

FuelSync – Complete Technical & Product Overview

****Generated:**** 2026-05-19
****Author:**** Daniel Markman
****Live URL:**** foodaniel.danielmms.site

1. What Is FuelSync?

FuelSync is a ****local-first Progressive Web App (PWA)**** for athlete nutrition and training tracking. It was originally a server-backed app (Express + PostgreSQL on a home server), and has since been migrated to a fully offline-capable PWA with no traditional backend. All user data lives in the browser's IndexedDB – no account, no password, no cloud.

The target user is a ****hybrid athlete**** – someone who does both running and strength training and needs intelligent macro cycling to match their daily training type.

2. Architecture

Before (Legacy – home server)

- ****Frontend:**** Vite + React → nginx static, served via Docker on home PC
- ****Backend:**** Node.js / Express (port 3002)
- ****Database:**** PostgreSQL 16 via Docker
- ****Auth:**** JWT tokens (30-day), stored in `localStorage` under key `fs\_token`
- ****AI:**** Gemini API called from backend Express route `/food/describe`
- ****Deploy:**** `docker compose build foodaniel && docker compose up -d foodaniel`

Now (Current – local-first PWA)

- ****Frontend:**** Static PWA deployed to ****GitHub Pages**** via GitHub Actions
- ****Data:**** ****IndexedDB**** (Dexie.js) – food logs, profile, PIN state, all local
- ****AI:**** ****Cloudflare Worker**** (`fuelsync-ai`) proxies Gemini API – hides the API key
- ****Strava:**** ****Cloudflare Worker**** (`fuelsync-strava`) handles OAuth callback, token refresh, stats proxy
- ****Auth:**** ****4-digit PIN**** (no accounts). 15 wrong attempts = full data wipe
- ****Deploy trigger:**** Push to `master` → GitHub Actions → GitHub Pages
- ****Domain:**** CNAME `foodaniel.danielmms.site` → `danielmarkmanCS.github.io`

Workers

Worker	File	Purpose
`fuelsync-ai`	`workers/ai/index.ts`	Gemini proxy (describe food, estimate macros, image analysis, meal suggestions)
`fuelsync-strava`	`workers/strava/index.ts`	Strava OAuth callback + token refresh + stats proxy
`fuelsync-sync`	`workers/sync/index.ts`	(in progress) background sync worker

The AI worker implements a ****model cascade****: tries `gemini-2.5-flash` → `gemini-2.0-flash` → `gemini-2.0-flash-lite` → `gemini-2.5-flash-lite`, skipping on 429/404 errors. Supports up to 3 API keys for rate-limit balancing.

3. Data Storage (IndexedDB via Dexie.js)

****File:**** `apps/web/src/lib/db.ts`

| Table | Contents |

```
|-----|-----|
| `profile` | User body stats, display name, activity level, goal mode, calorie
target |
| `food_logs` | Every food logged – name, calories, protein, carbs, fat, weight,
meal type, date, micronutrients |
| `pin_state` | Hashed PIN, attempt count, lockout timestamp |
```

All CRUD goes through `apps/web/src/api/localFood.ts` which wraps Dexie queries.

4. Authentication – PIN System

Files: `apps/web/src/lib/pin.ts`, `apps/web/src/screens/PinScreen.tsx`,
`apps/web/src/screens/AuthScreen.tsx`

- On first launch: user completes **2-step onboarding** (body stats + PIN creation)
- On return: PIN entry screen gate before app loads
- PIN is **hashed** before storage (never stored plain)
- **15 wrong attempts** triggers a complete local data wipe (IndexedDB cleared)
- Auth state in Zustand store (`authStore.ts`) uses a `pinVerified` boolean flag
- no JWT, no server

5. Screens

HomeScreen (`apps/web/src/screens/HomeScreen.tsx`)

The main dashboard. Shows:

- **Calorie hero** – large count-up animated calorie number, progress ring
- **Macro bars** – protein, carbs, fat as progress bars with targets
- **Micronutrient panel** – fiber, cholesterol, sodium, vitamin C, vitamin D, calcium, iron vs. gender-based targets
- **Recovery Score** – calculated from weekly run load + strength sessions + activity level. Labels: FRESH / ACTIVE / BUILDING / LOADED / OVERLOADED
- **Training Picker** – select today's training type (Rest / Cardio / Strength / Hybrid)
- **Strava Card** – recent run stats fetched via Cloudflare Worker
- **Weather Banner** – OpenWeatherMap API, shown when weather affects training
- **Time-of-day gradient header** – changes color from morning (blue) → midday (deep blue) → evening (purple) → night (near-black)

FoodScreen (`apps/web/src/screens/FoodScreen.tsx`)

Food logging hub. Features:

- **Three log modes:**
 1. **AI Smart** – free-text description → Gemini → auto-fills form
 2. **Manual** – name + weight + macros entered directly
 3. **Barcode scan** – looks up product via Open Food Facts API
- **Image analysis** – photo of food → Gemini vision → estimated macros
- **Meal suggestions** – AI suggests what to eat based on context (morning / pre-workout / post-workout / rest / evening)
- **Open Food Facts search** – search food database by name
- **Meal types:** Breakfast, Pre-Workout, Lunch, Post-Workout, Dinner, Snack (auto-detected from time of day)
- **Meal calorie targets** – per-meal calorie budgets that can be customized
- **Recent foods** – quick re-log from history
- **Favorites** – star any food for fast access
- **Meal templates** – save a group of foods as a reusable template
- **Recipes / Recipe Builder** – build multi-ingredient recipes, log the whole thing
- **Diary notes** – freetext daily note per date
- **Soft delete / undo** – items can be removed and then un-removed within the session

```
### HistoryScreen (`apps/web/src/screens/HistoryScreen.tsx`)
Browse past days. Shows per-day macro summary, expandable to see individual food logs. Handles soft-deleted entries (shows them as removed, with undo).
```

```
### ProfileSetupScreen (`apps/web/src/screens/ProfileSetupScreen.tsx`)
Edit body stats: weight, height, age, gender, activity level, display name. Also where the user signs out (clears PIN + data).
```

```
### GoogleAuthScreen (`apps/web/src/screens/GoogleAuthScreen.tsx`)
Handles Google Sign-In via Capacitor (native Android). Bridges the native Google auth flow into the app.
```

```
### PinScreen (`apps/web/src/screens/PinScreen.tsx`)
4-digit PIN entry. Shows attempt counter, locks out after 15 failures with full data wipe.
```

6. Nutrition Engine

```
**File:** `apps/mobile/src/services/nutritionEngine.ts` (shared via
`@mobile/services/nutritionEngine`)
```

```
### Macro Cycling Logic
Macros are calculated based on the user's **training type for the day**:
```

Training Type	Strategy
Rest	High fat, low carb (fat-adaptation / recovery)
Strength	High protein focus
Cardio	High carb (fuel for aerobic work)
Hybrid	Balanced macros

```
### Validation Rules
Before a food log is accepted, these checks run:
- Calories must match `P×4 + C×4 + F×9` within ±12%
- Total macros (P+C+F) must be ≤ `food weight × 1.1`
- No single macro can be >95% of the food's weight
- All values must be non-negative
```

Recovery Score Formula

```
runLoad = Σ(km × 1.5 × paceMultiplier)
strengthLoad = sessions × 15
score = max(5, 100 - min(100, (runLoad + strengthLoad) / activityMultiplier))
```

Pace multiplier: <4.5 min/km = 1.5×, <5.5 = 1.25×, <7.0 = 1.0×, slower = 0.75×
Activity multiplier: sedentary=0.4, light=0.65, moderate=1.0, very_active=1.7, extra_active=2.4

7. AI Features

All AI calls go to the `fuelsync-ai` Cloudflare Worker which proxies Gemini.

Feature	Endpoint	Description
Food description → macros	`POST /describe`	Free text ("chicken breast 200g with rice") → JSON macros
Weight-based estimate	`POST /estimate`	Food name + weight → macro estimate
Image analysis	`POST /analyze-image`	Base64 photo → macro estimate

| Meal suggestion | `POST /suggest` | Context (time/training) → suggested meal with macros |

The worker supports **multi-key rotation** (up to 3 Gemini API keys) and a **model cascade** to maximize uptime.

8. Strava Integration

Cloudflare Worker: `workers/strava/index.ts`
Frontend: `apps/web/src/api/strava.ts`
Component: `apps/web/src/components/StravaCard.tsx`

- OAuth2 flow – user authorizes, Worker handles callback and stores tokens in IndexedDB
- Worker proxies: recent activities, weekly stats, athlete profile
- Tokens are refreshed automatically via Worker when expired
- Stats feed into Recovery Score calculation

9. Open Food Facts Integration

File: `apps/web/src/api/openFoodFacts.ts`

- Text search: find products by name
- Barcode lookup: scan product barcode → instant macro fill
- Returns `OFFProduct` type with full nutrient data

10. Design System

FuelSync uses a custom dark "Space" design system:

Token	Value	Use
Background	`#0E1117`	Page background
Surface	`#161B27`	Cards
Surface 2	`#1D2333`	Nested cards
Edge	`rgba(255,255,255,0.07)`	Card borders
Text	`#DCE6FF`	Primary text
Muted	`#5A6990`	Labels, secondary
Blue	`#1E40DC`	Primary CTA
Blue 2	`#4B6FFF`	Hover / highlight
Green	`#05C56B`	Success, progress
Orange	`#FF8B00`	Warning
Purple	`#8034E0`	Special / PRs
Cyan	`#00BDD0`	Fresh / info
Red	`#EF3340`	Danger, overloaded

Font: **Inter** (400–900 weight), tight tracking. No italics.

Interactions: `nrc-press` = `scale(0.96)` on `:active`.

Animations: `nrc-a1` through `nrc-a6` = staggered slide-up entrance.

11. PWA & Mobile

- **VitePWA plugin** in `apps/web/vite.config.ts` generates service worker + manifest
- **Capacitor** (`apps/web/capacitor.config.ts`) wraps the PWA as a native Android app

- ****Native Google Sign-In**** via Capacitor plugin (GoogleAuthScreen)
- `apps/web/android/` – Android project generated by Capacitor
- Icon files needed: `apps/web/public/icon-192.png`, `icon-512.png`
- Sound on food log: Web Audio API C5→E5→G5 chord
(`apps/web/src/utils/sounds.ts`)

12. State Management

****Zustand stores:****

Store	File	Manages
`authStore`	`store/authStore.ts`	`pinVerified` flag, user profile object
`nutritionStore`	`store/nutritionStore.ts`	`todayLog`, `targets`, `weeklyLoad`

****Custom hook:**** `hooks/useNutrition.ts` – orchestrates `logDay`, macro targets, weekly load, weather, Strava refresh.

13. Key Libraries

Library	Purpose
React + Vite	UI framework + bundler
Zustand	Lightweight state management
Dexie.js	IndexedDB ORM
Capacitor	Native Android wrapper
VitePWA	Service worker + PWA manifest
Cloudflare Workers	AI + Strava serverless proxy

14. Deployment

Frontend (GitHub Pages)

- Push to `master` → GitHub Actions runs `vite build` → deploys to GitHub Pages
- Required GitHub secrets: `VITE\_AI\_WORKER\_URL`, `VITE\_STRAVA\_WORKER\_URL`,
`VITE\_OPENWEATHER\_KEY`
- DNS: Cloudflare CNAME `foodaniel.danielmms.site` → `danielmarkmanCS.github.io`

Workers (Cloudflare)

```
```bash
cd workers/ai && wrangler deploy
cd workers/strava && wrangler deploy
```
```

Secrets to set via Wrangler: `GEMINI\_API\_KEY`, `STRAVA\_CLIENT\_ID`,
`STRAVA\_CLIENT\_SECRET`

Legacy Docker (still active on home server)

```
```bash
cd /mnt/data/projects/mmswebsite
docker compose build foodaniel && docker compose up -d foodaniel
```
```

15. File Map Summary

...

/mnt/data/FuelSync/

| | | |
|------------------------|--|--|
| apps/web/ | | |
| src/ | | |
| screens/ | | |
| HomeScreen.tsx | - dashboard: calories, macros, recovery, | |
| FoodScreen.tsx | - log food: AI, manual, barcode, image, | |
| HistoryScreen.tsx | - browse past days | |
| ProfileSetupScreen.tsx | - body stats, settings | |
| AuthScreen.tsx | - onboarding: profile + PIN setup | |
| PinScreen.tsx | - PIN entry gate | |
| GoogleAuthScreen.tsx | - Capacitor Google Sign-In | |
| api/ | | |
| localFood.ts | - IndexedDB CRUD for food logs | |
| auth.ts | - local profile CRUD | |
| client.ts | - workerFetch helper (no axios) | |
| strava.ts | - Strava via Cloudflare Worker | |
| openFoodFacts.ts | - food search + barcode | |
| syncClient.ts | - sync worker client | |
| store/ | | |
| authStore.ts | - pinVerified, profile | |
| nutritionStore.ts | - todayLog, targets, weeklyLoad | |
| lib/ | | |
| db.ts | - Dexie schema | |
| pin.ts | - PIN hash/verify/lockout/wipe | |
| recentFoods.ts | - recent + favorites | |
| mealTemplates.ts | - save/load meal templates | |
| recipes.ts | - recipe builder | |
| diaryNotes.ts | - per-day notes | |
| mealCalTargets.ts | - per-meal calorie budgets | |
| customTargets.ts | - custom macro overrides | |
| components/ | | |
| StravaCard.tsx | | |
| TrainingPicker.tsx | | |
| WeatherBanner.tsx | | |
| hooks/useNutrition.ts | - main data hook | |
| utils/sounds.ts | - log sound: C5→E5→G5 | |
| capacitor.config.ts | - Capacitor native config | |
| android/ | - generated Android project | |
| workers/ | | |
| ai/index.ts | - Gemini proxy Worker | |
| strava/index.ts | - Strava OAuth Worker | |
| sync/index.ts | - sync Worker | |
| shared/types.ts | - MacroTargets, TrainingType, LoggedRun, | |
| etc. | | |
| ... | | |
| --- | | |

End of document.